Ministerul Educaț iei s i Cercetării al Republicii

Moldova **Universitatea Tehnică a Moldovei**

Facultatea Calculatoare, Informatică s i

Microelectronică Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Building and Exploring xv6

Elaborated by:
st. gr. FAF-242
Biriucov Alexandr

Verified by:
Reițman Patricia

Chis inău – 2026

1. Introduction

The purpose of this laboratory work is to become familiar with xv6, a small Unix-like teaching operating system.

During the laboratory work, xv6 was built and launched using QEMU, basic Unix commands were executed inside the xv6 shell, and the source code was inspected in order to understand the difference between user programs and kernel code.

The laboratory work also demonstrated how user programs interact with the kernel through system calls.

2. Building and Running xv6

The xv6 source code was located in:

```
~/xv6-riscv
```

The RISC-V compiler and QEMU emulator were installed in the Linux environment. The versions used during the laboratory work were:

```
riscv64-linux-gnu-gcc 13.3.0
QEMU emulator version 8.2.2
```

The operating system was built and started using:

```
make qemu
```

After successful compilation, xv6 booted and displayed:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

This confirmed that the xv6 kernel was running correctly and that the user shell had been started.

3. Using xv6 as the Unix It Is

Several commands were executed inside the xv6 shell:

```
ls
cat README
echo hello xv6
```



```
ls | grep c
wc README
```

3.1. Observations

1. Three programs included with xv6 are:

    • ls

    • cat

    • echo

2. For a pipe such as

    ```
    ls | grep c
    ```

    to work, the operating system must support **processes** and **inter-process com munication**.

    The shell runs the commands as separate processes and connects the standard out put of the first process to the standard input of the second process using a pipe.

3. The xv6 shell is much simpler than a modern Linux shell such as Bash, but basic Unix commands and concepts such as ls, cat, echo, pipes, and standard input/out put work in a similar way.

3.2. Example Command Output

The ls command displayed programs available inside xv6:

```
cat
echo
forktest
grep
init
kill
ln
```

```
ls
mkdir
rm
sh
stressfs
usertests
grind
wc
```



```
zombie
logstress
forphan
dorphan
sync
```

The command

```
echo hello xv6
```

produced:

```
hello xv6
```

4. Reading the Source

The source code of user programs and kernel system-call handlers was inspected using commands such as:

```
sed -n ’1,40p’ user/ls.c
sed -n ’1,30p’ user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c
```

4.1. System Calls Used by cat

The program user/cat.c uses several system calls.

- read() asks the kernel to read data from a file descriptor and place it into a buffer.
- write() asks the kernel to write data from a buffer to a file descriptor. • open() asks the kernel to open a file.
- close() asks the kernel to close an opened file descriptor.

• exit() terminates the current process.

The main data-processing loop in cat is:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
        fprintf(2, "cat: write error\n");
        exit(1);
    }
}
```


The program repeatedly reads data from a file descriptor and writes that data to standard output.

4.2. Kernel Implementation

The implementation of sys_read was found in:

```
kernel/sysfile.c
```

The function starts at line:

```
69: sys_read(void)
```

The implementation of sys_write starts at line:

```
83: sys_write(void)
```

This demonstrates that a user-level function such as read() ultimately causes the operating system kernel to execute the corresponding system-call implementation.

4.3. Kernel and User Space

The directories kernel/ and user/ contain different types of code.

- kernel/ contains privileged operating-system code that manages system resources and implements system calls.
- user/ contains ordinary programs that run outside the kernel and request operating system services through system calls.

This separation protects the kernel and system resources from direct access by user programs.

5. Creating a User Program

A new user program called sleep was added to xv6.

The source file was created as:

```
user/sleep.c
```

Its source code is:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
```



```
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
```

In the version of xv6 used during this laboratory work, the system-call interface for suspending a process for a number of clock ticks is named pause(). The program was added to the UPROGS section of the Makefile:

```
$U/_sleep\
```

After rebuilding xv6 using:

```
make clean
make qemu
```

the new program became available inside the xv6 shell.

5.1. Testing

The program was tested using:

```
sleep 10
```

The shell paused temporarily and then returned to the prompt.

Calling the program without the required argument:

```
sleep
```

produced the expected error message:

```
usage: sleep <ticks>
```

This confirmed that the program was compiled, added to the xv6 filesystem, and executed successfully.

6. Conclusion

During this laboratory work, the xv6 operating system was successfully built and executed using the QEMU RISC-V emulator.

Basic Unix commands were used to explore the xv6 user environment, including com mands involving pipes and standard input/output.

The source code of xv6 was then examined to understand the separation between user programs and kernel code and to observe how user programs access operating-system functionality through system calls.

Finally, a new user-level program, sleep, was created, added to the xv6 build system, compiled, and successfully executed.

The laboratory work provided a practical introduction to the internal structure of a Unix-like operating system and demonstrated the relationship between user programs, system calls, and kernel implementations.